

3 - Functions

May 12, 2026

Hàm (functions) trong Python

TS. Lê Thành Trung

May 12, 2026

Contents

1	Mục tiêu bài học	3
2	Cơ bản về hàm (function) trong Python	3
2.1	Khái niệm	3
2.2	Khi nào nên tách thành hàm?	3
2.3	Cú pháp gọi hàm	3
2.4	Hàm được xây dựng sẵn trong Python	3
2.4.1	Nhóm hàm thường dùng	3
2.4.2	Lỗi thường gặp với hàm dựng sẵn	4
2.5	Hàm tự định nghĩa	4
3	Biến cục bộ và biến toàn cục	5
3.1	Biến cục bộ (local variable)	5
3.2	Biến toàn cục (global variable)	5
3.3	Thay đổi giá trị biến truyền vào hàm	6
3.4	Lỗi thường gặp	7
4	Hàm lồng nhau	7
5	Hàm Lambda	8
6	Hàm là biến truyền vào của một hàm khác	9
7	Viết mô tả cho hàm	9
8	Tổng kết	11
9	Bài tập	11

1 Mục tiêu bài học

- Hiểu vai trò của hàm trong tổ chức chương trình.
- Sử dụng thành thạo các hàm dựng sẵn thường gặp.
- Tự định nghĩa hàm với tham số và giá trị trả về.
- Phân biệt biến cục bộ và biến toàn cục.
- Viết hàm lồng nhau, hàm lambda, và truyền hàm làm đối số.
- Rèn luyện tư duy chia nhỏ bài toán theo mô-đun hàm.

2 Cơ bản về hàm (function) trong Python

2.1 Khái niệm

- Hàm (function) là một khối lệnh có tên, thực hiện một nhiệm vụ cụ thể.
- Mục đích chính:
 - Tái sử dụng mã nguồn.
 - Giảm lặp code.
 - Tăng khả năng đọc, kiểm thử và bảo trì.

2.2 Khi nào nên tách thành hàm?

- Một đoạn xử lý được dùng lặp lại từ 2 lần trở lên.
- Một logic dài làm hàm chính khó đọc.
- Muốn đặt tên rõ nghĩa cho một bước xử lý (ví dụ: `validate_email`, `calculate_gpa`).

2.3 Cú pháp gọi hàm

`ten_ham(danh_sách_biến_tương_ứng)`

```
[1]: print(len("Python"))  
      print(abs(-12))  
      print(round(3.14159, 2))
```

```
6  
12  
3.14
```

2.4 Hàm được xây dựng sẵn trong Python

2.4.1 Nhóm hàm thường dùng

- Xử lý số: `abs`, `round`, `pow`, `min`, `max`, `sum`.
- Xử lý chuỗi/tập dữ liệu: `len`, `sorted`, `reversed`, `enumerate`, `zip`.
- Chuyển kiểu: `int`, `float`, `str`, `list`, `tuple`, `set`, `dict`.
- I/O cơ bản: `print`, `input`.

Lưu ý: - Cần phân biệt *hàm làm thay đổi dữ liệu truyền vào* và *hàm trả về dữ liệu mới*. - Cần đọc mô tả chi tiết hàm, đặc biệt là kiểu trả về để tránh lỗi logic.

```
[2]: nums = [7, -2, 15, 4]
print("len:", len(nums))
print("min:", min(nums))
print("max:", max(nums))
print("sum:", sum(nums))
print("sorted:", sorted(nums))
print("abs(-9):", abs(-9))
print("round(3.14159, 3):", round(3.14159, 3))
```

```
len: 4
min: -2
max: 15
sum: 24
sorted: [-2, 4, 7, 15]
abs(-9): 9
round(3.14159, 3): 3.142
```

```
[4]: nums = [7, -2, 15, 4]
new_nums = sorted(nums)
print("Original:", nums)
print("Sorted:", new_nums)

nums.sort()
print("Sorted in-place:", nums)
```

```
Original: [7, -2, 15, 4]
Sorted: [-2, 4, 7, 15]
Sorted in-place: [-2, 4, 7, 15]
```

2.4.2 Lỗi thường gặp với hàm dựng sẵn

- Quên gọi hàm: viết `len` thay vì `len(data)`.
- Nhầm giữa `sorted(data)` và `data.sort()`.
- Truyền sai kiểu dữ liệu, ví dụ `sum("123")` gây lỗi.
- Xác định sai kiểu dữ liệu trả về.

2.5 Hàm tự định nghĩa

```
def ten_ham(tham_so_1, tham_so_2, ..., tham_so_n=gia_tri_mac_dinh):
    # thân hàm
    return gia_tri
```

- Từ khóa `def` để khai báo hàm.
- `return` kết thúc hàm và trả kết quả (nếu không có thì trả về `None`).
- Tên hàm nên dùng `snake_case` và bắt đầu với động từ.
- Ví dụ: viết hàm tính diện tích hình chữ nhật

```
[6]: def tinh_dien_tich_hinh_chu_nhat(chieu_dai, chieu_rong):
    dien_tich = chieu_dai * chieu_rong
```

```

    return dien_tich

print(tinh_dien_tich_hinh_chu_nhat(5, 3))

```

15

- Ví dụ: viết hàm tính doanh thu = số lượng x đơn giá x (1 + phần trăm VAT), với phần trăm VAT mặc định là 10%.

```

[7]: def tinh_doanh_thu(gia_ban, so_luong, VAT=0.1):
    doanh_thu = gia_ban * so_luong * (1 + VAT)
    return doanh_thu

print(tinh_doanh_thu(100, 5))
print(tinh_doanh_thu(100, 5, VAT=0.15))

```

550.0

575.0

3 Biến cục bộ và biến toàn cục

3.1 Biến cục bộ (local variable)

- Được tạo bên trong hàm.
- Chỉ dùng được trong phạm vi hàm đó.
- Biến local khác hàm sẽ không nhìn thấy nhau.

3.2 Biến toàn cục (global variable)

- Khai báo bên ngoài mọi hàm.
- Có thể đọc từ trong hàm.
- Muốn sửa giá trị global bên trong hàm thì dùng từ khóa `global`.

```

[12]: x = 100 # global

def demo_scope():
    y = x + 10 # local
    z = 20 # local
    print("Trong ham, y =", y)
    print("Trong ham, z =", z)

demo_scope()
print("Ngoai ham, x =", x)

```

Trong ham, y = 110

Trong ham, z = 20

Ngoai ham, x = 100

```
[18]: x = 100 # global
def thay_doi_x():
    x = 200 # local
    return x

print(thay_doi_x())
```

200

```
[19]: x = 100 # global
def thay_doi_x():
    x += 200 # local
    return x

print(thay_doi_x())
```

```
-----
UnboundLocalError                                Traceback (most recent call last)
Cell In[19], line 6
      3     x += 200 # local
      4     return x
----> 6 print(thay_doi_x())

Cell In[19], line 3, in thay_doi_x()
      2 def thay_doi_x():
----> 3     x += 200 # local
      4     return x

UnboundLocalError: cannot access local variable 'x' where it is not associated
↳ with a value
```

```
[20]: x = 100 # global
def thay_doi_x():
    global x
    x += 200 # local
    return x

print(thay_doi_x())
```

300

3.3 Thay đổi giá trị biến truyền vào hàm

- Python **không** chia rạch ròi như C++ (và một số ngôn ngữ khác) thành truyền tham trị (pass-by-value) và truyền tham chiếu (pass-by-reference).
- Python dùng truyền tham chiếu đối tượng (**call by object reference** hay **call by sharing**).

Hiểu ngắn gọn: - Hàm nhận tham chiếu đến cùng đối tượng. - Với đối tượng **immutable** (int,

float, str, tuple), không thay đổi được đối tượng truyền vào. - Nếu đối tượng là **mutable** (list, dict, set) và sửa **tại chỗ**, bên ngoài sẽ thấy thay đổi. - Nếu bên trong hàm chỉ **gán lại** tham số sang đối tượng mới, bên ngoài **không đổi**.

```
[21]: def tang_so(n):  
        n += 1  
  
n = 5  
tang_so(n)  
print("n sau khi dung ham tang so:", n)
```

n sau khi dung ham tang so: 5

```
[23]: def them_phan_tu(ds):  
        ds.append(999)  # list la mutable -> sua tai cho  
  
def gan_lai_danh_sach(ds):  
        ds = [0, 0, 0]  # chi doi bien cuc bo ds, khong doi list ben ngoai  
  
b = [1, 2, 3]  
them_phan_tu(b)  
print("Sau them_phan_tu, b =", b)  
  
print("\nTruoc gan_lai_danh_sach, b =", b)  
gan_lai_danh_sach(b)  
print("Sau gan_lai_danh_sach, b =", b)  # van giu nguyen list cu
```

Sau them_phan_tu, b = [1, 2, 3, 999]

Truoc gan_lai_danh_sach, b = [1, 2, 3, 999]

Sau gan_lai_danh_sach, b = [1, 2, 3, 999]

3.4 Lỗi thường gặp

- Cố truy cập biến local từ ngoài hàm gây **NameError**.
- Quên **global** khi muốn cập nhật biến toàn cục.
- Lạm dụng biến global làm chương trình khó kiểm soát.
- Khuyến nghị: ưu tiên truyền tham số và trả kết quả thay vì phụ thuộc global.

4 Hàm lồng nhau

- Hàm lồng nhau là hàm được định nghĩa bên trong một hàm khác.
- Thường dùng để:
 - Che giấu logic phụ trợ (helper).
 - Tạo closure (ghi nhớ trạng thái): một hàm con được trả về nhưng vẫn nhớ các biến của hàm cha, dù hàm cha đã chạy xong.

```
[27]: def xu_ly_danh_sach(nums):
        def la_so_chan(x):
            return x % 2 == 0

        ket_qua = [n for n in nums if la_so_chan(n)]
        return ket_qua

print(xu_ly_danh_sach([1, 2, 3, 4, 5, 6]))
```

[2, 4, 6]

```
[29]: def tao_ham_nhan(k):
        def nhan(x):
            return x * k
        return nhan

nhan_2 = tao_ham_nhan(2)
nhan_5 = tao_ham_nhan(5)
print(nhan_2(10))
print(nhan_5(10))
```

20

50

5 Hàm Lambda

- lambda là hàm ẩn danh, thường dùng cho biểu thức ngắn.
- Cú pháp: `lambda tham_so: bieu_thuc`.

Khi nên dùng: - Viết nhanh hàm đơn giản dùng 1 lần. - Truyền vào `sorted`, `map`, `filter`.

Không nên dùng: - Logic phức tạp nhiều nhánh (nên dùng `def`).

```
[31]: square = lambda x: x * x
print(square(6))
```

36

```
[36]: students = [("An", 8.2), ("Binh", 7.5), ("Chi", 9.1)]
students_sorted = sorted(students, key=lambda item: item[1], reverse=True)
print(students_sorted)
```

[('Chi', 9.1), ('An', 8.2), ('Binh', 7.5)]

```
[38]: nums = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, nums))
double = list(map(lambda x: x * 2, nums))
print(evens)
print(double)
```



```
[2, 4, 6]
[2, 4, 6, 8, 10, 12]
```

6 Hàm là biến truyền vào của một hàm khác

- Trong Python, hàm là đối tượng hạng nhất (first-class object).
- Có thể truyền hàm vào hàm khác, trả hàm từ hàm khác.

```
[40]: def apply_operation(x, y, op):
        return op(x, y)

def cong(a, b):
    return a + b

def nhan(a, b):
    return a * b

print(apply_operation(3, 4, cong))
print(apply_operation(3, 4, nhan))
print(apply_operation(3, 4, lambda a, b: a ** b))
```

```
7
12
81
```

```
[43]: def process_scores(scores, transformer):
        return [transformer(s) for s in scores]

scores = [6.5, 7.0, 8.0, 9.0]
scores_10 = process_scores(scores, lambda s: s * 10 / 10)
scores_bonus = process_scores(scores, lambda s: min(s + 0.5, 10))
print(scores_10)
print(scores_bonus)
```

```
[6.5, 7.0, 8.0, 9.0]
[7.0, 7.5, 8.5, 9.5]
```

7 Viết mô tả cho hàm

Mô tả cho hàm (docstring) là đoạn văn bản đặt ngay dưới dòng `def` để giải thích hàm làm gì.

Mục đích: - Giúp người khác đọc code nhanh hơn. - Giúp chính mình dễ bảo trì khi quay lại code cũ. - Công cụ như `help()` có thể hiển thị mô tả này.

Mẫu mô tả ngắn gọn nên có: - Chức năng chính của hàm (1 câu). - Ý nghĩa các tham số đầu vào. - Giá trị trả về.

Mẫu docstring gợi ý:

```
def ten_ham(tham_so_1, tham_so_2):
    """Tóm tắt chức năng của hàm.

    Args:
        tham_so_1 (kiểu dữ liệu): Ý nghĩa tham số 1.
        tham_so_2 (kiểu dữ liệu): Ý nghĩa tham số 2.

    Returns:
        (kiểu dữ liệu) Mô tả giá trị trả về.
    """
```

Lưu ý: - Viết ngắn, rõ, dùng từ nhất quán. - Không mô tả điều hiển nhiên kiểu “gán biến x”. - Nếu hàm đơn giản, chỉ cần 1 dòng mô tả cũng đủ.

```
[46]: def doi_do_C_sang_F(do_C):
        """Chuyển nhiệt độ từ độ C sang độ F.

        Args:
            do_C (số thực): Nhiệt độ theo thang đo C.

        Returns:
            (số thực): Nhiệt độ theo thang đo F theo công thức  $F = C * 9 / 5 + 32$ 
        """
        return do_C * 9 / 5 + 32

    print(doi_do_C_sang_F(30))
```

86.0

```
[47]: print(doi_do_C_sang_F.__doc__)
```

Chuyển nhiệt độ từ độ C sang độ F.

```
Args:
    do_C (số thực): Nhiệt độ theo thang đo C.

Returns:
    (số thực): Nhiệt độ theo thang đo F theo công thức  $F = C * 9 / 5 + 32$ 
```

```
[48]: help(doi_do_C_sang_F)
```

Help on function doi_do_C_sang_F in module __main__:

```
doi_do_C_sang_F(do_C)
    Chuyển nhiệt độ từ độ C sang độ F.
```

```
Args:
    do_C (số thực): Nhiệt độ theo thang đo C.
```

Returns:

(số thực): Nhiệt độ theo thang đo F theo công thức $F = C * 9 / 5 + 32$

8 Tổng kết

- Hàm giúp chia nhỏ bài toán và tái sử dụng code.
- Hàm dựng sẵn giúp xử lý dữ liệu nhanh và an toàn.
- Hàm tự định nghĩa cần rõ tham số, kiểu dữ liệu, giá trị trả về.
- Nhớ chắc phạm vi biến để tránh lỗi khó tìm.
- `lambda` phù hợp cho xử lý ngắn gọn, không thay thế hoàn toàn `def`.
- Có thể truyền hàm làm đối số để viết code linh hoạt hơn.
- Nên viết mô tả chi tiết cho hàm tự định nghĩa và đọc mô tả hàm trước khi sử dụng.

9 Bài tập

1. Viết hàm `greet(name)` trả về chuỗi: Xin chào, <name>!.
2. Viết hàm `hinh_chu_nhat(dai, rong)` trả về cả chu vi và diện tích.
3. Viết hàm `doi_do_F(do_f)` đổi từ độ F sang độ C theo công thức: $F = C * 9 / 5 + 32$.
4. Viết hàm `lam_tron_diem(score)` trả về điểm làm tròn 1 chữ số thập phân.
5. Viết hàm `sap_xep_tang(ds)` trả về danh sách mới đã sắp xếp tăng dần (không sửa danh sách gốc).
6. Viết hàm `tao_ham_cong(k)` trả về một hàm mới để cộng số đầu vào với `k` (closure).